SIMATS ENGINEERING

CO4-ASSESSMENT TOOL3- INTEGRATED EXPERIMENTS

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA1709

Name	**DHANUSH S**
Register Number	**192524317**

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

Time Duration: 45 Minutes

QUESTIONS: 2 **Total Marks:20**

Code of Conduct:

*I **Dhanush S (192524317)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

DHANUSH S
(192524317)

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

(a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.

(b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification

(c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, −1 each step, −5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α, γ, ε).

(b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.

(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

EXPERIMENT 1: DECISION TREE CLASSIFICATION

Aim

To implement and evaluate a Decision Tree classifier using the Iris dataset and classify the iris flowers into their respective classes.

Objective

To:

1. **Load and preprocess the dataset.**
2. **Display the first five rows and data types.**
3. **Split the dataset into training and testing sets.**
4. **Build a Decision Tree using the Gini Index.**
5. **Display the tree structure and identify the root node.**
6. **Evaluate the model using accuracy, confusion matrix and classification report.**
7. **Identify at least two misclassified instances.**

(a) Dataset Loading, Preprocessing and Splitting

The Iris dataset contains 150 observations and four input features:

- **Sepal Length**
- **Sepal Width**
- **Petal Length**
- **Petal Width**

The target classes are:

- **Setosa**
- **Versicolor**
- **Virginica**

The dataset does not contain missing values, so no missing-value treatment is required. The target labels are converted to class names.

The dataset is divided into:

- **80% training data**
- **20% testing data**

Python Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import pandas as pd

# Load Iris dataset
iris = load_iris(as_frame=True)

df = iris.frame.copy()

# Convert numerical target into class names
df["target"] = df["target"].map(
    dict(enumerate(iris.target_names))
)

# Display first 5 rows
print("First 5 rows:")
print(df.head())

# Display data types
print("\nData Types:")
print(df.dtypes)

# Separate features and target
X = df[iris.feature_names]
y = df["target"]

# Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
```

```
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))

# Build Decision Tree using Gini Index
model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)

model.fit(X_train, y_train)

# Display tree
print("\nDecision Tree Structure:")
print(export_text(model, feature_names=list(X.columns)))

# Predictions
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy)

# Confusion Matrix
cm = confusion_matrix(
    y_test,
    y_pred,
    labels=iris.target_names
)

print("\nConfusion Matrix:")
print(cm)

# Classification Report
print("\nClassification Report:")
```

```
print(
    classification_report(
        y_test,
        y_pred
    )
)

# Misclassified instances
results = X_test.copy()
results["Actual"] = y_test
results["Predicted"] = y_pred

misclassified = results[
    results["Actual"] != results["Predicted"]
]

print("\nMisclassified Instances:")
print(misclassified)
```

Output – First Five Rows

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

```
Data Types
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
target               object
The dataset is split into:
Training samples: 120
Testing samples: 30
```

(b) Decision Tree and Root Node

The Decision Tree is constructed using the Gini Index as the splitting criterion.

The important portion of the generated tree is:

|--- petal length (cm) <= 2.45

| |--- class: setosa

|

|--- petal length (cm) > 2.45

| |--- petal width (cm) <= 1.65

| | |--- ...

| |

| |--- petal width (cm) > 1.65

| |--- ...

Root Node

The root node is petal length (cm) <= 2.45.

Justification

The root node is selected because the Decision Tree algorithm chooses the feature and threshold that provide the best reduction in impurity according to the Gini criterion.

Petal length provides a very strong separation between the Iris classes, particularly between Setosa and the other two classes.

Therefore:

Root node = Petal Length ≤ 2.45 cm

(c) Model Evaluation

The model gives the following accuracy:

Accuracy = 0.9333

Therefore, the classification accuracy is approximately:

93.33%

Confusion Matrix

Using the order:

Setosa, Versicolor, Virginica

the confusion matrix is:

[[10 0 0]

[0 9 1]

[0 1 9]]

Interpretation:

Actual Class	Correctly Classified	Misclassified
Setosa	10	0

Versicolor	9	1
Virginica	9	1

Setosa is classified perfectly, while one Versicolor and one Virginica sample are incorrectly classified.

Classification Report

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.90	0.90	0.90	10
virginica	0.90	0.90	0.90	10
accuracy			0.93	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

Misclassified Instances

Two misclassified test instances are:

Sample	Actual Class	Predicted Class
Sample 134	Virginica	Versicolor
Sample 77	Versicolor	Virginica

Performance Comment

The Decision Tree achieved approximately 93.33% accuracy, which indicates good classification performance.

Setosa was classified with 100% accuracy because its petal measurements are clearly different from the other classes. The errors occurred between Versicolor and Virginica because their feature values overlap.

Result

The Iris dataset was successfully classified using a Decision Tree with the Gini Index. The root node was found to be Petal Length ≤ 2.45 cm, and the model achieved approximately 93.33% accuracy on the test dataset.

EXPERIMENT 2: REINFORCEMENT LEARNING – Q-LEARNING

Aim

To implement the Q-Learning algorithm for a 4×4 Grid World environment and observe the learned optimal policy.

Objective

To:

1. **Define a 4×4 Grid World.**
2. **Define states and actions.**
3. **Initialize the Q-table with zeros.**
4. **Set the learning parameters α, γ and ε.**
5. **Run Q-Learning for at least 100 episodes.**
6. **Record Q-values at Episodes 1, 50 and 100.**
7. **Extract the final learned policy.**
8. **Plot cumulative reward per episode.**
9. **Analyze the convergence behaviour.**

(a) Environment Definition

The Grid World contains 16 states arranged as:

```
+------+------+------+------+
|  0   |  1   |  2   |  3   |
+------+------+------+------+
|  4   |  5   |  6   |  7   |
+------+------+------+------+
|  8   |  9   | 10   | 11   |
+------+------+------+------+
| 12   | 13   | 14   | 15   |
+------+------+------+------+
```

Where:

- **State 0 = Start**
- **State 15 = Goal**
- **States 5 and 6 = Obstacles**

Actions

The agent can perform four actions:

0 = Up

1 = Down

2 = Left

3 = Right

Reward Structure

As specified in the question:

- **Reaching the goal = +10**
- **Each normal step = −1**
- **Moving into an obstacle / invalid movement = −5**

Q-Table

The Q-table contains:

16 states × 4 actions

and is initially filled with zeros.

Hyperparameters

Learning rate (α) = 0.1

Discount factor (γ) = 0.9

Initial exploration rate (ε) = 1.0

Minimum ε = 0.05

ε decay = 0.995

Number of episodes = 100

Q-Learning Formula

The Q-value is updated using:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\,[r + \gamma \max Q(s',a') - Q(s,a)]$$

where:

- **s = current state**
- **a = current action**
- **r = reward**
- **s' = next state**
- **α = learning rate**
- **γ = discount factor**

Python Program

```
import numpy as np
import random
import matplotlib.pyplot as plt

# Reproducibility
random.seed(42)
np.random.seed(42)

# Grid size
ROWS = 4
COLS = 4
NUM_STATES = 16

# Start and goal
START = 0
GOAL = 15

# Obstacles
OBSTACLES = {5, 6}

# Actions
ACTIONS = ["Up", "Down", "Left", "Right"]

# Movement directions
MOVES = {
    "Up": (-1, 0),
    "Down": (1, 0),
    "Left": (0, -1),
    "Right": (0, 1)
}

# Hyperparameters
alpha = 0.1
gamma = 0.9
```

```
epsilon = 1.0
epsilon_min = 0.05
epsilon_decay = 0.995

episodes = 100

# Q-table initialized with zeros
Q = np.zeros((NUM_STATES, len(ACTIONS)))



# Environment function
def step(state, action_index):

    row, col = divmod(state, COLS)

    action = ACTIONS[action_index]
    dr, dc = MOVES[action]

    new_row = row + dr
    new_col = col + dc

    # Invalid movement
    if not (0 <= new_row < ROWS and
            0 <= new_col < COLS):
        return state, -5, False

    next_state = new_row * COLS + new_col

    # Obstacle
    if next_state in OBSTACLES:
        return state, -5, False

    # Goal
    if next_state == GOAL:
        return next_state, 10, True
```

```
    # Normal movement
    return next_state, -1, False


# Store cumulative rewards
rewards_per_episode = []

# Store Q-values at required episodes
recorded_q = {}

# Q-Learning
for episode in range(1, episodes + 1):

    state = START
    total_reward = 0

    for step_count in range(100):

        # Epsilon-greedy action selection
        if random.random() < epsilon:
            action = random.randrange(len(ACTIONS))
        else:
            action = np.argmax(Q[state])

        # Take action
        next_state, reward, done = step(state, action)

        # Q-learning update
        target = reward

        if not done:
            target += gamma * np.max(Q[next_state])

        Q[state, action] += alpha * (
            target - Q[state, action]
        )
```

```
        total_reward += reward
        state = next_state

        if done:
            break

    rewards_per_episode.append(total_reward)

    # Record Q-values at Episodes 1, 50 and 100
    if episode in [1, 50, 100]:
        recorded_q[episode] = (
            Q[0].copy(),
            Q[1].copy()
        )

    # Reduce exploration
    epsilon = max(
        epsilon_min,
        epsilon * epsilon_decay
    )


# Display Q-values
for episode in [1, 50, 100]:

    print("\nEpisode", episode)

    print("State 0:")
    for i in range(4):
        print(ACTIONS[i], "=", round(
            recorded_q[episode][0][i], 2
        ))

    print("State 1:")
    for i in range(4):
```

```
        print(ACTIONS[i], "=", round(
            recorded_q[episode][1][i], 2
        ))


# Extract final policy
print("\nFinal Learned Policy:")

for row in range(ROWS):

    for col in range(COLS):

        state = row * COLS + col

        if state == GOAL:
            symbol = "G"

        elif state in OBSTACLES:
            symbol = "X"

        else:
            best_action = np.argmax(Q[state])
            symbol = ACTIONS[best_action][0]

        print(f"{symbol:^7}", end="")

    print()


# Display final Q-table
print("\nFinal Q-Table:")
print(np.round(Q, 2))


# Plot cumulative reward
plt.figure(figsize=(10, 5))
```

```
plt.plot(
    range(1, episodes + 1),
    rewards_per_episode
)

plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning: Cumulative Reward per Episode")
plt.grid(True)

plt.show()
```

(b) Q-Table Evolution

For two representative states, State 0 and State 1, the recorded Q-values are:

Episode	State	Up	Down	Left	Right
1	0	-2.66	-0.41	-1.40	-0.27
1	1	0.00	0.00	-0.30	0.00
50	0	-7.06	-2.20	-6.97	-2.65
50	1	-6.25	-6.24	-2.76	-1.60
100	0	-5.32	1.10	-5.10	-1.71
100	1	-6.16	-5.92	-1.87	0.17

The Q-values change as the agent explores the environment and learns which actions provide better long-term rewards.

Interpretation

At the beginning, the Q-table contains little useful information because it is initialized with zeros. As the agent interacts with the environment, rewards are propagated backward through the Q-table.

By Episode 100, the Q-values for actions leading toward the goal become more favorable than actions that lead toward obstacles or unnecessary movement.

(c) Final Learned Policy

The final learned policy obtained from the Q-table is:

D R R D

D X X D

R D D D

R R R G

Where:

U = Up

D = Down

L = Left

R = Right

X = Obstacle

G = Goal

Thus, starting from State 0, the learned agent follows a path toward the goal while avoiding the obstacle states.

One possible learned path is:

0 → 4 → 8 → 9 → 10 → 11 → 15

Cumulative Reward Plot

The Python program generates a graph with:

- **X-axis = Episode number**
- **Y-axis = Cumulative reward**

The graph shows how the reward obtained by the agent changes during training.

Convergence Behaviour

During the early episodes, the agent performs considerable exploration because ε is initially high. Therefore, the cumulative rewards can fluctuate considerably.

As training continues:

1. **The agent learns the effect of different actions.**
2. **Q-values for useful actions increase.**
3. **The agent increasingly selects actions that lead toward the goal.**
4. **Exploration decreases because ε decays.**
5. **The learned policy becomes more stable.**

The reward curve may not increase smoothly in every episode because Q-Learning still performs exploration and the agent can occasionally choose a poor action. Therefore, convergence should be interpreted from the overall trend and the stability of the learned policy rather than from every individual episode.

Result

Q-Learning was successfully implemented for a 4×4 Grid World. The Q-table was initialized with zeros and updated for 100 episodes using the specified reward structure. The final Q-table was used to extract the optimal policy, and the cumulative reward per episode was plotted to observe the learning and convergence behaviour.